Strictly Dominated: A Post-Mortem of a Spectral–Associative Language Model, and the In-Block Causality Leak Behind Its 7.2× Headline

Akhilesh Gogikar*

July 8, 2026

Abstract

We report a negative result. Over several months we developed AKI (*Associative Koopman Imagination*) and SKC (*Spectral Koopman Capsules*), a family of attention-replacement language-model blocks combining blockwise Walsh–Hadamard (WHT) spectral mixing, associative capsule memory, a Koopman-style latent rollout, and an episodic "engram" sidecar. Early experiments showed a headline result of 7.2× lower bits-per-byte than a matched Transformer (0.2455 vs. 1.7712 BPB). We show forensically that this result, and every below-baseline number produced by the family, is explained by an in-block causality leak: a transform that is causal *between* 64-position blocks but dense *within* them, so that each position reads inputs from its own future. A minimal harness demonstrates that (a) perturbing a future in-block position moves earlier outputs, and (b) because two WHT applications with a learned diagonal between them form a dyadic (XOR) convolution containing the permutation $t \mapsto t \oplus 1$, a two-line model trained on i.i.d. tokens drives validation BPB to 2.07 — far below the information-theoretic floor of $\log_2 V = 4.0$ — by literally reading the next token at every even position. The only strictly causal repair (lower-triangular masking of the block operator) destroys the fast transform and the entire advantage. With the leak fixed, matched-pair runs show the architecture was *strictly dominated*: +0.094 BPB worse than a parameter/data/seed-matched Transformer at the longest horizon while running 1.3–5.7× slower per step. We dissect the methodology failures that let a leak survive months of development — proxy-FLOPs "fairness" contracts, 10-minute micro-gates, multi-knob attribution chaos, stability-via-damping, and CI gates calibrated to the contaminated number — and distill checks that would have caught it in a day.

1 Introduction

Sub-quadratic replacements for attention are an active research area, and spectral token mixing is a recurring ingredient. This note documents a research program (AKI/SKC) whose central empirical support was an artifact of a causality violation, and whose leak-free variants never beat a matched Transformer on quality or speed. We publish it because (i) the failure mechanism — a *blockwise* fast transform that looks causal at the block level while leaking within blocks — is easy to reintroduce in any FFT/WHT-style mixer for autoregressive models, and passes shape-level and loss-goes-down sanity checks; (ii) the exploit mechanism is subtle (spectral-domain gating composes

*Reproduction: the full leak harness runs in ∼35 s on CPU via `python3 experiments/wht_causality/confirm_wht_unsalvageable.py` in the accompanying repository: `https://github.com/Akhilesh-Gogikar/causal-certificate`.

into a future-reading permutation, so the leak is not just present but *trainable*); and (iii) the process failures are generic to compute-constrained architecture research.

Contributions:

1. A forensic, reproducible demonstration of the in-block WHT causality leak and of its exploitability by SGD (Section 3);
2. Matched-pair evidence that the strictly causal architecture is dominated on the quality/speed plane (Section 4);
3. A methodology post-mortem with concrete, low-cost gates — including a five-minute i.i.d.-token oracle test that bounds any causal model at $\log_2 V$ BPB — that we argue should precede any headline claim for a new sequence mixer (Section 5);
4. **Our central original contribution:** a numeric *perturbation-based strict-causality certificate* — inject perturbations at future positions $t' > t$ and require exact numerical invariance of all outputs at positions $\leq t$ — packaged as a standalone, architecture-agnostic CI tool (`causal-certificate`). Unlike shape-level or mask-inspection checks, it certifies the realized forward computation, and it is what ultimately detected the leak described here.

2 The architecture family, briefly

First, the names. **AKI** stands for *Associative Koopman Imagination*: *associative* because token content is stored and retrieved via outer-product (Hebbian-style) key–value bindings rather than pairwise attention; *Koopman* because latent state is advanced by a learned, approximately linear operator in a lifted feature space, in the spirit of Koopman operator theory for nonlinear dynamics; and *imagination* because that operator is rolled forward several steps *without consuming new tokens*, producing predicted future latents that condition the current output. **SKC** stands for *Spectral Koopman Capsules*: *spectral* because token mixing happens in a Walsh–Hadamard transform domain rather than by dot-product attention; *capsules* because the associative memory is organized into a bank of small, independently gated slots (each a low-rank key–value store) rather than one flat matrix. **AKF** (*Associative Koopman Flow*) names the latent rollout module itself, and **Engram** — borrowing the neuroscience term for a physical memory trace — is an episodic sidecar that reads before it writes, storing compressed summaries of past context outside the main residual stream.

The AKI/SKC family (twelve versions, V0–V12) shares one invariant: replace dense self-attention with a strict-causal structured state path. The load-bearing primitive is `causal_wht_blockwise`: reshape the sequence into blocks of 64 positions and apply a normalized Walsh–Hadamard butterfly within each block; blocks are combined by a causal decaying scan. Layers use the pattern *WHT → elementwise/gated ops in the spectral domain → WHT back*, alongside an associative capsule bank (SKC), a Koopman-flow latent rollout (AKF), and a read-before-write episodic memory (Engram). Full version genealogy and claim boundaries are preserved in the repository (`docs/architecture/AKI_VERSIONS_DETAILED.md`).

3 The causality leak

3.1 Mechanism

The blockwise WHT applies the full 64×64 Hadamard butterfly across all positions of each block. It is causal *between* blocks and dense *within* them: the output at position t mixes inputs at $t' > t$

up to the end of t's block. The blockwise framing is precisely why it survived review: block-level dataflow diagrams look causal, and the GPU path (a Triton kernel computing the same dense Hx per block) inherits the identical leak.

3.2 Test A: future perturbations move earlier outputs

Perturbing in-block position t'=40 changes the output at positions $t < 40$ by up to 0.125 under the repository implementation, and by exactly 0.0 under the lower-triangular-masked variant — the only strictly causal repair.[1]

3.3 Test B: the leak is not just present but trainable

Why did the leak convert into (fictitiously) excellent language modeling? Two WHT applications with a per-position diagonal s between them — exactly the layer shape used throughout the family — compute $H\,\mathrm{diag}(s)\,H$, a *dyadic (XOR) convolution*. Choosing s to be the Hadamard spectrum of δ_1 yields $y[t] = x[t \oplus 1]$: at every even position the mixer outputs the embedding of the *next* token. SGD finds this optimum readily. On i.i.d. uniform tokens with V=16, where any causal model is information-theoretically pinned at $\log_2 16 = 4.0$ BPB:

Mixer	val BPB	vs. floor 4.0
Repository blockwise WHT (leaky)	**2.0721**	impossible without future access
Lower-triangular-masked WHT	4.0021	at floor
No WHT (diagonal only)	4.0029	at floor

Table 1: I.i.d.-token oracle test. The leaky mixer reads token t+1 perfectly at even positions (≈ 2.0 BPB = half the positions at 0 bits, half at 4). Deeper stacks of the same primitive reach ≈ 0 BPB.[3]

This explains the provisional-patent-era headline (0.2455 vs. 1.7712 BPB, "7.2×"): the model was grading itself on an open-book exam.

3.4 No causal-and-fast-and-spectral repair exists

Masking the future entries of the block operator ($H \mapsto \mathrm{tril}(H)$) restores causality but destroys the butterfly factorization, the $O(B \log B)$ cost, and empirically the entire advantage: clean strictly causal re-runs were +0.09 to +0.24 BPB *worse* than baseline.[4]

4 Matched evaluation of the leak-free architecture

With strict causality enforced, paired runs trained the AKI-family architecture and a Transformer under one harness with matched parameters, optimizer, data, and seed (`fairness_contract.json`):

Two readings matter. First, *quality*: the deficit at 10k steps is stable (`gap_trend.tsv`), so this is not a transient. Second, *speed*: the architecture was pitched as sub-quadratic-therefore-faster, yet measured 1.3–5.7× slower per step at practical sequence lengths — the serial scan, folded rollout, sidecar memory, and gate machinery cost more wall-clock than the dense attention they replaced. The architecture was on the wrong side of the quality/speed frontier in every matched run: strictly dominated.

[1] Evidence: `experiments/wht_causality/report.json (regenerated; torch 2.2.2, seed-pinned)`

[4] Evidence: `docs/contribution_fwht_causality.md; docs/architecture/POST_MORTEM_WHY_NON_SOTA.md`

run	steps	AKI BPB	xfmr BPB	ΔBPB	AKI ms/step	xfmr ms/step	slowdown
`aki_vs_transformer_10k`	10000	1.6576	1.5635	+0.094	170.6	89.7	1.9×
`...125m_fastpath_d512`	4351	1.6555	1.5004	+0.155	494.4	392.7	1.3×
`aki_l32_gate_1500`	1401	2.3277	2.0198	+0.308	121.6	64.6	1.9×
`aki_d480_h8_speed_gate`	1451	1.9888	2.0188	−0.030	207.8	64.4	3.2×
`...aki30pct_d480_lowrank`	1451	1.9920	2.0208	−0.029	368.9	64.3	5.7×

Table 2: Matched pairs (params/optimizer/data/seed). The only "wins" are −0.03 BPB at ∼1450 steps — inside optimization noise and bought at 3–6× step cost. At the longest horizon the deficit is +0.094 BPB against a ∼ 2× faster baseline, and the gap trend is stable, not closing.[6]

Root cause, in one sentence: every mechanism in the family mixes the past through a fixed-width, content-agnostic bottleneck, while attention's content-based addressing over an unbounded cache is exactly what byte-level language modeling rewards at these scales.

5 How a leak survives months: methodology post-mortem

1. **Proxy-FLOPs fairness was fictional.** The contract used proxy_flops = 6 · params · tokens, which assumes a Transformer compute profile and is blind to serial per-token cost. "Matched FLOPs" hid a 1.3–5.7× wall-clock disadvantage. *Gate on wall-clock on identical hardware.*

2. **A mis-calibrated baseline manufactured false hope.** An early "16× convergence" headline compared against a Transformer sitting at cross-entropy ≈ 5.08 after 800 steps — barely below the $\ln 256 \approx 5.55$ random floor for bytes, i.e. an under-tuned baseline. Months of roadmap chased this artifact. *Sanity-check the baseline against known-good curves before comparing.*

3. **Micro-gate horizons.** Architectures were screened at 10-minute / 1–2k-step gates where init and LR-schedule transients dominate; the only "wins" in Table 2 live there, and vanish by 10k steps.

4. **Attribution chaos.** Runs toggled Engram, speculator, capsules, MoE, feedback, kernels, halting, and schedules simultaneously; no run isolated the architecture, so no gain was attributable and no loss localizable.

5. **Stability-via-damping removed the thesis.** Each instability was met with a clamp (gate max 0.35, correction radius 0.30, control scale 0.015, engine gain 0.20), converging the novel block toward "a small residual tweak on the backbone" — quietly deleting the mechanism that was supposed to win.

6. **The contaminated number became a CI gate.** The benchmark contract enforces `max_eval_bpb: 0.35` — a threshold only a leaky model can meet at this scale — institutionalizing the artifact as the definition of success.[7]

7. **The check that catches it in five minutes.** Train the smallest model containing the new mixer on i.i.d. uniform tokens. Any validation BPB below $\log_2 V$ is proof of non-causality (Table 1). We recommend this oracle as a standing pre-benchmark gate for any proposed sequence mixer, alongside a perturbation probe (Test A) run on *both* the reference and accelerated kernel paths.

[7] Evidence: `configs/benchmarks/skc_vs_mamba_vs_transformers.json`

6 Related work

Non-causal spectral token mixing is well established for *encoders* (e.g. Fourier-mixing architectures); the failure documented here arises specifically when such mixers are ported to autoregressive decoding behind a block-causal wrapper. State-space and long-convolution models obtain causal spectral mixing correctly via causal kernels/scans; our result is not evidence against those designs, only against unmasked in-block fast transforms. While ad-hoc probes of causal masking appear in engineering folklore, we are not aware of prior work formalizing a numeric perturbation-invariance *certificate* of strict causality as a reusable, model-agnostic CI gate; we claim the certificate, and the perturbation analysis it packages, as original contributions of this work and release them at `https://github.com/Akhilesh-Gogikar/causal-certificate`. We keep citations minimal deliberately: an internal audit found earlier agent-generated prior-art lists unreliable, and we cite only what we have verified.

7 Limitations and scope

Results are at small scale (byte-level, $\leq$125M-parameter class, sequence lengths where dense attention is fast); we do not claim the family is dominated at all scales, only that no evidence supports the original claims. The matched-run table is reproduced from the repository post-mortem; raw logs for three of the five rows predate the current branch, and we are regenerating the headline pair and the Triton-path leak confirmation on commodity cloud GPUs [to regenerate: RunPod matched gate + GPU/Triton Test A, this revision].

8 Conclusion

The publishable value of this program is the autopsy: a precise, reproducible mechanism by which a blockwise fast transform silently reads the future, a demonstration that spectral gating makes such leaks *trainable* rather than merely present, matched evidence that the honest architecture lost on both axes, and a five-minute oracle test that converts this class of error from a months-scale trap into a pre-flight check.

Reproducibility. `experiments/wht_causality/` regenerates Tables 1 and Test A on CPU in $\sim$35 s. Matched-gate reproduction instructions: `docs/RUNPOD_REPRO.md`.